Table of contents

Introduction **4**

Primary Data Types and OS Allocation **4**

i) Data Type Sizes Across Architectures 4

ii) Practical Example: Observing Data Type Sizes 5

OS Memory Segments **5**

i) Memory Segment Overview 5

ii) Memory Layout Visualization 6

Practical Code Examples **6**

i) Observing variable addresses 6

ii) Pointer and memory allocation 7

Laboratory Exercises **7**

i) Lab 1: Data Type Size Exploration 7

ii) Lab 2: Memory Segment Address Mapping 8

Summary and Key Takeaways **11**

Knowledge Check Question with Answers **12**

List of Figures

Figure 1: Program to observe data type sizes.....5

Figure 2: Process memory layout (high to low addresses).....6

Figure 3: Program to observe variable addresses.....6

Figure 4: Correct pointer and heap usage.....7

Figure 5: Linux command to create file1.c file and write C program on that file1.c file.....7

Figure 6: Linux command to view the contents (i.e C program that prints the size of char, int, float, double, long, pointer) inside file1.c file and the output of that linux command.....7

Figure 7: Linux command of 4 compilation stages to convert file1.c file to executable file "file1".....8

Figure 8: Linux command to show the program output of data type sizes on a Linux system and confirms successful execution with exit status 0.....8

Figure 9: Linux command to create file2.c file and write C program on that file2.c file.....8

Figure 10: Linux command to view the contents (i.e C program that prints variable addresses and calculates and prints the address difference between heap and stack) inside file2.c file.....9

Figure 11: Linux command of 4 compilation stages to convert file2.c file to executable file "file2"....9

Figure 12: Linux command to show the program output of variable addresses on a Linux system with address difference between stack and heap and confirms successful execution with exit status 0.....10

Figure 13: Linux command to create file3.c file and write contents inside file3.c file.....10

Figure 14: Linux command to view the contents (i.e C program that prints variable addresses) inside file3.c file.....10

Figure 15: Linux command of 4 compilation stages to convert file3.c file to executable file "file3"..10

Figure 16: Linux command to show the program output shows address of pointer and stack with value stored and confirms successful execution with exit status 0. And list files.....11

List of Table

Table I: Datatype sizes across system architecture..... 4
Table II: Process memory segments and their purposes..... 5

Introduction

Data types form the foundation of programming and operating system interaction. A data type is a contract between
programmer, compiler, and operating system that specifies:
1) The number of bytes to allocate in memory
2) How to interpret those bytes (integer, character, floatingpoint)
3) What operations are valid on that data
When a programmer declares a variable, the OS must
allocate physical or virtual memory to store that data. Understanding this relationship is crucial for writing efficient,
secure, and portable code.

Primary Data Types and OS Allocation

i) Data Type Sizes Across Architectures

Table I shows how data type sizes vary across different system architectures, directly impacting OS memory allocation.

Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
int	2 bytes	4 bytes	4 bytes	Affects pointer arithmetic
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

Table I: Datatype sizes across system architecture

ii) Practical Example: Observing Data Type Sizes

```
  GNU nano 8.7.1                  file1.c *
#include <stdio.h>

int main() {
        printf("=== Data Type Sizes on This System ===\n\n");

        printf("char: %zu bytes\n", sizeof(char));
        printf("int: %zu bytes\n", sizeof(int));
        printf("float: %zu bytes\n", sizeof(float));
        printf("double: %zu bytes\n", sizeof(double));
        printf("long: %zu bytes\n", sizeof(long));
        printf("pointer: %zu bytes\n", sizeof(int*));

        return 0;
}
```

Figure 1: Program to observe data type sizes.

OS Memory Segments

The operating system creates a process address space—avirtual view of memory divided into segments. Each segment serves a specific purpose and has different characteristics.

i) Memory Segment Overview

Segment	Purpose	Grows	Managed By
Text	Machine code (instructions)	Fixed	Compiler/Loader
Data	Initialized global/static variables	Fixed	Compiler/OS
BSS	Uninitialized global/static variables	Fixed	OS
Heap	Dynamically allocated memory	Upward	Programmer + OS
Stack	Local variables, function calls	Downward	Compiler + OS

Table II: Process memory segments and their purposes

ii) Memory Layout Visualization

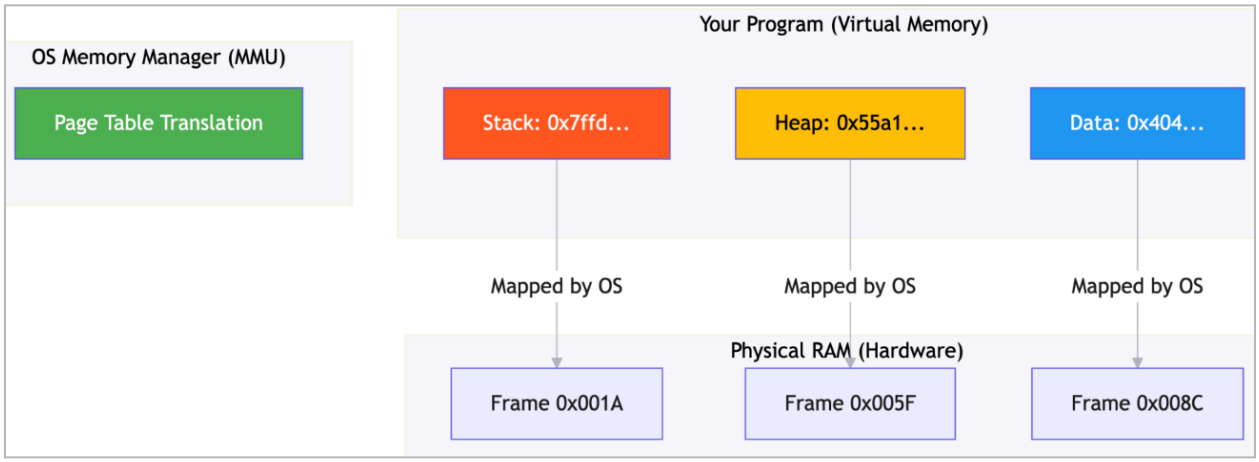


Figure 2: Process memory layout (high to low addresses).

Key Observation: Stack addresses are high (0x7ffd...), Heap addresses are mid-range (0x55a1...), and Data/BSS addresses are low (0x404...).

Practical Code Examples

i) Observing variable addresses

```
  GNU nano 8.7.1                                  file2.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 150; // Data segment
int bss_var; // BSS/Data segment

int main() {
        int local_var = 30; // Stack
        static int local_static = 20; // Data segment

        int *heap_var = (int *)malloc(sizeof(int)); //Heap*heap_var = 500;
        printf(".......Variable Addresses.......\n\n");
        printf("global_var: %p (Data/BSS)\n", (void*)&global_var);
        printf("bss_var: %p (Data/BSS)\n", (void*)&bss_var);
        printf("local_var: %p (Stack)\n", (void*)&local_var);
        printf("local_static: %p (Data/BSS)\n", (void*)&local_static);
        printf("heap_var ptr: %p (Stack - pointer)\n",(void*)&heap_var);
        printf("*heap_var: %p (Heap - data)\n", (void*)heap_var);

        free(heap_var);
        return 0;
}
```

Figure 3: Program to observe variable addresses

ii) Pointer and memory allocation

```
  GNU nano 8.7.1                                                file3.c
#include <stdio.h>
#include <stdlib.h>

int main() {
        int *forheap; // Declare pointer (lives on STACK)
        forheap = (int *)malloc(sizeof(int)); //Allocate memory on HEAP first!
        *forheap = 30; // store value to allocated memory// Print addresses
        printf("Address of pointer (STACK): %p\n", (void*)&forheap);
        printf("Address of data (HEAP): %p\n", (void*)forheap);
        printf("Value stored: %d\n", *forheap);

        return 0;
}
```

Figure 4: Correct pointer and heap usage

Laboratory Exercises

i) Lab 1: Data Type Size Exploration

```
root@DESKTOP-OAP8OK0:/batch38/Week4# touch file1.c
root@DESKTOP-OAP8OK0:/batch38/Week4# nano file1.c
```

Figure 5: Linux command to create file1.c file and write C program on that file1.c file.

```
root@DESKTOP-OAP8OK0:/batch38/Week4# cat file1.c
#include <stdio.h>

int main() {
        printf("=== Data Type Sizes on This System ===\n\n");

        printf("char: %zu bytes\n", sizeof(char));
        printf("int: %zu bytes\n", sizeof(int));
        printf("float: %zu bytes\n", sizeof(float));
        printf("double: %zu bytes\n", sizeof(double));
        printf("long: %zu bytes\n", sizeof(long));
        printf("pointer: %zu bytes\n", sizeof(int*));

        return 0;
}
```

Figure 6: Linux command to view the contents (i.e C program that prints the size of char, int, float, double, long, pointer) inside file1.c file and the output of that linux command.

```
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc -E file1.c -o file1.i
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc -S file1.i -o file1.s
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc -c file1.s -o file1.o
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc file1.o -o file1
```

Figure 7: Linux command of 4 compilation stages to convert file1.c file to executable file "file1".

```
root@DESKTOP-OAP8OK0:/batch38/Week4# ./file1
=== Data Type Sizes on This System ===

char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
long: 8 bytes
pointer: 8 bytes
root@DESKTOP-OAP8OK0:/batch38/Week4# echo $?
0
```

Figure 8: Linux command to show the program output of data type sizes on a Linux system and confirms successful execution with exit status 0.

From figure 8 we can see our Linux system on 64-bit architecture allocates the same space of 1, 4, 4, 8, 8, 8 byte for char, int, float, double, long, pointer respectively which matches the 64-bit column of Table I.

Question: Why might sizeof (long) differ between 32-bit and 64-bit systems?
Ans: "sizeof(long)" differs because 64-bit systems use larger word sizes to support bigger memory addresses, so "long" expands from 4 bytes to 8 bytes.

ii) Lab 2: Memory Segment Address Mapping

```
root@DESKTOP-OAP8OK0:/batch38/Week4# touch file2.c
root@DESKTOP-OAP8OK0:/batch38/Week4# nano file2.c
```

Figure 9: Linux command to create file2.c file and write C program on that file2.c file.

```
root@DESKTOP-OAP8OK0:/batch38/Week4# cat file2.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 150; // Data segment
int bss_var; // BSS/Data segment

int main() {
        int local_var = 30; // Stack
        static int local_static = 20; // Data segment

        int *heap_var = (int *)malloc(sizeof(int)); //Heap*heap_var = 500;
        printf(".......Variable Addresses.......\n\n");
        printf("global_var: %p (Data/BSS)\n", (void*)&global_var);
        printf("bss_var: %p (Data/BSS)\n", (void*)&bss_var);
        printf("local_var: %p (Stack)\n", (void*)&local_var);
        printf("local_static: %p (Data/BSS)\n", (void*)&local_static);
        printf("heap_var ptr: %p (Stack - pointer)\n",(void*)&heap_var);
        printf("*heap_var: %p (Heap - data)\n", (void*)heap_var);
        long diff = (long)((char*)heap_var - (char*)&local_var);
        printf("Address difference between stack and heap: %ld bytes\n", diff);

        free(heap_var);
        return 0;
}
```

Figure 10: Linux command to view the contents (i.e C program that prints variable addresses and calculates and prints the address difference between stack and heap) inside file2.c file.

```
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc -E file2.c -o file2.i
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc -S file2.i -o file2.s
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc -c file2.s -o file2.o
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc file2.o -o file2
```

Figure 11: Linux command of 4 compilation stages to convert file2.c file to executable file "file2".

```
root@DESKTOP-OAP8OK0:/batch38/Week4# ./file2
.......Variable Addresses.......

global_var: 0x5cff4d08f010 (Data/BSS)
bss_var: 0x5cff4d08f01c (Data/BSS)
local_var: 0x7ffe540181b4 (Stack)
local_static: 0x5cff4d08f014 (Data/BSS)
heap_var ptr: 0x7ffe540181b8 (Stack - pointer)
*heap_var: 0x5cff686e6010 (Heap - data)
Address difference between stack and heap: -38478269325732 bytes
root@DESKTOP-OAP8OK0:/batch38/Week4# echo $?
0
```

Figure 12: Linux command to show the program output of variable addresses on a Linux system with address difference between stack and heap and confirms successful execution with exit status 0.

```
root@DESKTOP-OAP8OK0:/batch38/Week4# touch file3.c
root@DESKTOP-OAP8OK0:/batch38/Week4# nano file3.c
```

Figure 13: Linux command to create file3.c file and write contents inside file3.c file.

```
root@DESKTOP-OAP8OK0:/batch38/Week4# cat file3.c
#include <stdio.h>
#include <stdlib.h>

int main() {
        int *forheap; // Declare pointer (lives on STACK)
        forheap = (int *)malloc(sizeof(int)); //Allocate memory on HEAP first!
        *forheap = 30; // store value to allocated memory// Print addresses
        printf("Address of pointer (STACK): %p\n", (void*)&forheap);
        printf("Address of data (HEAP): %p\n", (void*)forheap);
        printf("Value stored: %d\n", *forheap);

        return 0;
}
```

Figure 14: Linux command to view the contents (i.e C program that prints variable addresses) inside file3.c file.

```
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc -E file3.c -o file3.i
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc -S file3.i -o file3.s
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc -c file3.s -o file3.o
root@DESKTOP-OAP8OK0:/batch38/Week4# gcc file3.o -o file3
```

Figure 15: Linux command of 4 compilation stages to convert file3.c file to executable file "file3".

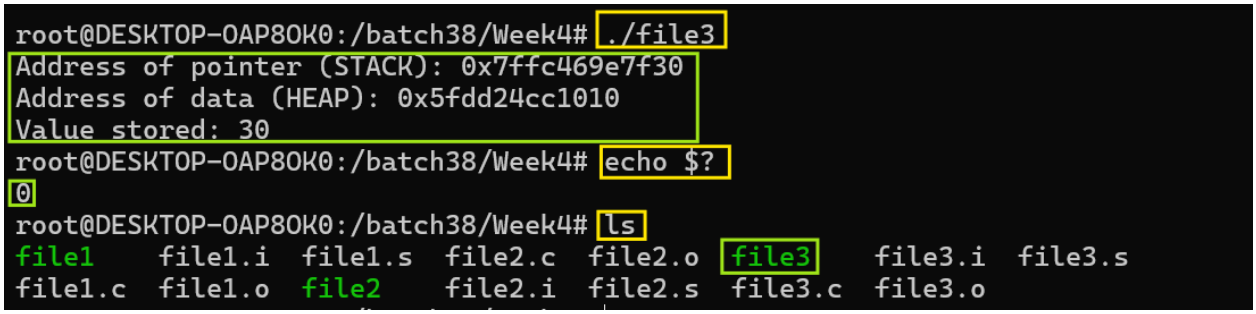


Figure 16: Linux command to show the program output where the output shows address of pointer and stack with value stored and confirms successful execution with exit status 0. And list files.

The output I observed from "file2 executable file" is that the program prints both the segment type and the hex address for each variable.I also observed that the address difference between stack and heap gives us a negative value of -38478269325732 bytes which proves stack address is numerically higher than the heap. We can also verify the stack is above heap manually by comparing the hex value output of "figure 12" where we can see local_var (Stack) = 0x7ffe540181b4, and *heap_var (Heap) = 0x5cff686e6010, this tells us that 0x7ffe540181b4 > 0x5cff686e6010 because of hex value, and larger hex means higher memory address, therefore the stack is at top. And in "file3 executable file" I observed the pointer variable itself is on the stack, but the memory it points to (where "30" is stored) is in the heap.

Summary and Key Takeaways

Data types are contracts — they tell the OS how much memory to allocate and how to interpret it.

• Memory is segmented — stack for locals, heap for dynamic, data/BSS for globals, text for code. (we will further explore the text section in more details if necessary)

• The OS manages it all — tracking, allocating, protecting, and virtualizing memory.

• Stack is automatic; heap is manual — choose based on lifetime and size needs.

• Virtual memory provides isolation — each process has its own address space.

• Pointers bridge stack and heap — the pointer variable lives on stack, but points to heap memory.

Knowledge Check Question with Answers

1. What is a data type in the context of programming and operating systems?

Ans: A data type is a contract that tells the OS how many bytes to allocate, how to interpret those bytes, and what operations are valid.

2. Name the five main memory segments in a process memory layout.

Ans: Text, Data, BSS, Heap, Stack.

3. What is the difference between the Data segment and the BSS segment?

Ans: Data segment stores initialized global/static variables, while BSS stores uninitialized global/static variables.

4. Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

Ans: To let the stack grow downward from high addresses and the heap grow upward from low/mid addresses without colliding, while keeping code (text) fixed at low addresses.

5. What happens to stack memory when a function returns? Who reclaims it?

Ans: The stack frame is automatically popped, and the compiler + OS reclaim the memory.

6. How does the OS know which memory belongs to which process?

Ans: Through virtual memory and page tables each process has its own address space mapped by the OS memory manager.

Github link: OS-Security-And-Networks/Week4/README.md at main · firstnamecheten/OS-Security-And-Networks